

2. 목적별 특화 프롬프트 설계 2

1. 역할 기반 프롬프팅 개요

역할 기반 프롬프팅이란 생성형 AI 모델(ChatGPT 등)에게 특정한 **역할**이나 **페르소나**를 부여하여 그에 맞는 어조와 정보를 담은 응답을 이끌어내는 프롬프트 기법입니다. 모델에게 마치 튜터, 비서, 상담사, 면접관 등의 역할을 **명시적으로 설정**하면, 일반적인 응답과는 달리 그 **역할에 특화된 표현 방식과 내용**을 갖춘 답변을 생성하게 됩니다. 이를 통해 **응답의 톤과 포커스**를 원하는 방향으로 조율할 수 있으며, 보다 **구체적이고 전문적인 답변**을 얻을 수 있습니다.

예를 들어, 아무 역할 지시 없이 질문하면 모델은 기본적으로 **친절한 일반 어시스턴트**처럼 대답합니다. 그러나 **“당신은 마케팅 전문가입니다”**와 같은 역할을 부여한 뒤 동일한 질문을 하면, **시장 분석**이나 **브랜딩 기법** 등 마케팅 관점에서 깊이 있는 답변을 하게 됩니다. **역할에 따라 말투와 표현도** 달라지는데, **“식품회사 마케팅 담당자”** 역할로 신제품 홍보 문구를 요청하면 **소비자의 구매 욕구를 자극**하는 마케팅 어조의 답변을 하고, **“온라인 쇼핑물 고객센터 상담원”** 역할로 불만 응대를 시키면 **객관적인 사실 전달**과 **고객 공감 및 문제 해결 노력**이 담긴 답변을 얻게 됩니다. 이처럼 **역할 설정**을 활용하면 모델의 **응답 스타일**을 과제나 상황에 맞게 맞춤화할 수 있습니다.

역할 기반 프롬프트는 다양한 상황에서 활용됩니다. 예를 들어 **튜터봇** 역할을 주면 교육자처럼 상세한 설명과 예시를 들며 가르쳐줄 수 있고, **비서봇** 역할을 주면 필요한 정보를 빠르게 정리해주거나 일정 관리처럼 **실용적인 도움**을 주는 식입니다. **상담사** 역할을 지정하면 공감 어린 조언이나 심리적인 지원 위주의 답변을, **면접관** 역할을 주면 면접 시 평가 관점에서의 조언이나 모의질문을 제공하는 등, 같은 질문이라도 역할에 따라 **응답의 구조와 초점**이 크게 달라집니다. 역할 기반 프롬프팅은 이러한 맥락 맞춤형 응답을 통해 AI와의 상호작용을 더욱 풍부하고 유용하게 만들며, 경우에 따라 **응답의 명확성 및 정확성 향상**에도 도움이 될 수 있다고 알려져 있습니다.

※ **참고:** 연구 결과에 따르면, **역할 부여**가 추론이나 설명이 필요한 작업에서 응답의 질을 높이는 사례도 보고되었습니다. 예를 들어 GPT-3.5 모델에 **수학 선생님 역할**을 부여하여 어려운 문제를 풀게 하자 정답률이 **53.5%에서 63.8%로 향상된 사례**가 있습니다. 다만 **사실 질의처럼 정확한 지식이 필요한 작업의 경우에는 역할 페르소나 추가만으로는 성능 향상이 보장되지 않는다**는 연구도 있습니다. 따라서 역할 프롬프팅은 응답의 맥락과 스타일을 조정하는 용도로 유용하며, 항상 정답률을 높이는 만능 해결책은 아니라는 점을 유의해야 합니다.

2. 역할별 프롬프트 설계 전략

역할 기반 프롬프팅을 효과적으로 사용하려면 **프롬프트 설계** 단계에서 역할 지시를 명확히 하고, 해당 역할에 어울리는 **어조와 형식**을 모델에게 알려주는 것이 중요합니다. 이를 구현하는 일반적인 방법은 **시스템 프롬프트** 또는 **유저 프롬프트**에 역할을 서술하는 것입니다.

- **시스템 메시지에 역할 지정:** OpenAI ChatCompletion API에서는 **system** 역할 메시지를 사용해 모델의 기본 페르소나를 설정합니다. 예를 들어 시스템 메시지에 **"당신은 친절하고 꼼꼼한 튜터봇으로서, 사용자의 글쓰기 능력 향상을 도와주세요"**라고 명시하면, 이후 유저의 질문에 모델이 해당 지시에 맞춰 튜터로서 답변하게 됩니다. 시스템 메시지는 사용자에게 보이지 않고 모델에게만 주어지는 지침이므로, **모델의 초기 역할과 톤을 세팅하는 데 적합합니다.**
- **유저 프롬프트에 역할 지시 포함:** 만약 별도의 시스템 메시지를 제공할 수 없는 환경(예: 일반 챗봇 UI)이라면, 사용자 메시지의 시작 부분에 **"(역할 설명) ..."**을 넣어 모델에게 **역할을 인지**시킬 수 있습니다. 예를 들어 유저가 **"당신은 이제부터 취업 상담사입니다. 다음 질문에 답해 주세요: ..."**라고 입력하면 모델은 해당 역할을 부여받고 답변하게 됩니다. 이 방식은 간편하지만, 모델이 **역할 지시와 실제 질문을 혼동하지 않도록 명료하게 구분**하는 표현(예: **"당신은 ... 역할입니다."** 형태)를 사용하는 것이 좋습니다.

역할 프롬프트 작성 시 고려사항:

- **역할의 구체화:** 가능한 한 구체적이고 분명한 역할을 정의하세요. 단순히 **"전문가"**보다 **"5년 경력의 프론트엔드 개발자"**처럼 **역할의 전문 분야나 관점을 상세히 설명**하면 모델이 더 명확한 맥락을 갖고 답변합니다. **역할의 책임이나 목표도 함께 제시**하면 좋습니다 (예: **"...로서 사용자가 어려워하는 개념을 쉽게 설명해주는 것이 역할입니다"**).
- **어조(Tone)와 말투 설정:** 역할에 따라 원하는 말투나 어조가 있다면 프롬프트에 포함시킵니다. 예를 들어 **"친절하고 인내심 있는 상담사"**, **"공손하고 전문적인 비서"**, **"엄격하지만 유머러스한 멘토"** 등으로 어조를 묘사하면 모델이 답변에서 그 느낌을 반영합니다. 필요하다면 **1인칭/3인칭 화법**이나 전문 용어 사용 정도도 지시할 수 있습니다.
- **정보량과 형식:** 역할이 답변에 포함해야 할 정보의 양과 형식을 지정합니다. 튜터봇이라면 **단계별 설명**이나 **예제**를 요구하고, 비서라면 **간결한 요약**이나 **목록 형태 정리**를 지시하는 식입니다. 예를 들어 시스템 메시지에 **"답변은 bullet point 목록으로 제시하세요"** 또는 **"근거를 들어 설명하세요"**와 같은 형식 지침을 추가하면, 역할에 맞는 형식으로 응답하게 됩니다.
- **모델 제한과 정책 준수:** 역할을 부여할 때 OpenAI 이용정책을 벗어나는 **위험한 역할** (예: **"해커"**, **"범죄 공모자"**)이나 **부적절한 페르소나**는 피해야 합니다. 또한 **지나치게 친밀한 개인 역할**(예: 가족, 연인 등)은 예기치 않은 감정적인 응답이나 편향된 답변을 유발할 수 있으므로 가급적 피하고, 보다 **중립적이고 전문적인 역할**을 사용하는 것이 좋습니다.

다. 역할 묘사 시 성별 등 편향 요소를 포함하지 않거나 중립적으로 표현하는 것이 바람직합니다.

- **두 단계 프롬프팅(고급):** 역할 몰입을 강화하는 고급 기법으로 **두 단계 역할 부여**가 있습니다. 먼저 **Role-Setting** 프롬프트로 모델에게 역할을 주고 그에 대한 응답(역할 수락 혹은 자기소개 형식의 출력)을 확인한 다음, 이어서 실제 질문을 하는 방법입니다. 예를 들어:

1. 사용자: "당신은 이제 10년 경력의 수학 교사입니다. 이 역할을 이해했다면 그렇게 행동하겠다고 답해주세요."

2. 모델(Assistant): "네, 저는 10년 경력의 수학 교사입니다. 어떤 수학 문제를 도와드릴까요?"

3. 사용자: "이제 문제가 뭐냐면..." (실제 질문 진행)

이런 식으로 모델이 **자신의 역할을 한 번 명확히 인식**하도록 한 뒤 본 답변을 유도하면, 복잡한 작업일수록 역할을 더 충실히 반영한 답변을 얻을 수 있다는 보고도 있습니다. 다만 이러한 방법은 호출 횟수가 늘어나 토큰 소모가 많아지므로, 꼭 필요한 경우에 활용합니다.

요약하면, **역할 기반 프롬프트 설계**에서는 "**누구처럼 대답해야 하는가**"를 모델에 분명히 알려주고, 말투나 세부 표현까지 안내함으로써 **일관되고 목적에 맞는 응답**을 얻도록 하는 것이 핵심 전략입니다.

3. Playground 실습: 역할별 응답 비교

이제 동일한 질문에 대해 **다양한 역할**을 부여했을 때 답변이 어떻게 달라지는지 직접 실습해 보겠습니다. OpenAI의 Playground(또는 ChatGPT 인터페이스)를 활용하여, 한 가지 질문을 가지고 **튜터봇, 상담사, 면접관, 비서** 역할을 각각 지정한 뒤 결과를 비교해 볼 것입니다.

##실습 2.1. OpenAI Playground 실습

실습 질문: "**자기소개서를 잘 쓰는 법을 알려줘**"

(취업 준비생이 묻는 상황을 가정하고, 자기소개서를 효과적으로 작성하는 방법에 대해 조언을 구하는 질문입니다.)

실습 방법:

- Playground에서 모델은 기본적으로 GPT-4o-mini를 사용합니다 (필요하면 GPT-4o로 변경 가능). 우선 **시스템 메시지**에 역할을 설정하고, **사용자 메시지**로 질문을 입력하는 방식을 사용하겠습니다. Playground 상단의 "System" 필드에 해당 역할 지시를 넣고, "User" 필드에 위의 질문을 입력한 후 "**Send**"를 눌러 응답을 얻습니다. 이 과정을

역할별로 반복해 보세요. (만약 시스템 필드를 사용할 수 없다면, 사용자 메시지에 먼저 역할 지시를 한 문장 넣은 후 질문을 이어서 작성해도 됩니다.)

테스트할 역할 시나리오:

1. **튜터봇 역할:** 시스템 메시지에 "당신은 글쓰기 과외 선생님입니다. 체계적이고 이해하기 쉽게 글쓰는 요령을 가르쳐주세요." 와 같이 입력합니다.
2. **상담사 역할:** 시스템 메시지에 "당신은 공감적이고 따뜻한 커리어 상담사입니다. 취업 준비생의 고민을 듣고 조언해주세요." 와 같이 입력합니다.
3. **면접관 역할:** 시스템 메시지에 "당신은 인사 담당 면접관입니다. 지원자의 자기소개서에 대해 평가와 조언을 해주세요." 와 같이 입력합니다.
4. **비서봇 역할:** 시스템 메시지에 "당신은 업무 능률이 뛰어난 비서입니다. 간결하고 실행 가능한 취업 조언을 제공하세요." 와 같이 입력합니다.

각 설정 후 **User 메시지**에는 공통으로 "자기소개서를 잘 쓰는 법을 알려줘" 를 입력합니다. 이렇게 총 네 번 질의하여 네 가지 역할의 답변을 확보합니다.

예상 응답 특징: (실제로 모델이 어떻게 답했는지 비교해 보며 아래 사항을 검토하세요)

- **튜터봇:** 학생을 가르치듯이 **체계적인 설명**과 함께 자기소개서 작성의 핵심을 알려줄 것입니다. 예를 들면 번호를 매긴 **단계별 가이드**, 좋은 예시와 나쁜 예시를 들며 이유를 설명하는 등 **교육자적인 어조**를 기대할 수 있습니다.
- **상담사:** 먼저 "자기소개서 작성이 부담될 수 있다는 것을 이해한다"는 식으로 공감을 표시할 수 있습니다. 이후 **위로와 응원**의 말투로 조언을 건네고, 지원자의 강점을 찾는 방법이나 멘탈 관리 등 **정서적인 측면**까지 다룰 가능성이 높습니다. 답변은 부드럽고 **격려하는 느낌**일 것입니다.
- **면접관:** 자기소개서를 보는 **채용자의 관점**에서 직설적이고 현실적인 피드백을 줄 수 있습니다. 예를 들어 "면접관으로서 이런 점을 높이 평가한다"거나 "이런 실수는 피해야 한다"는 등, **평가자 시각**의 조언이 나올 것입니다. 경우에 따라 모의 질문을 던져 생각해 보게 할 수도 있습니다. 전반적으로 **전문적이고 단호한 어조**가 예상됩니다.
- **비서봇:** 불필요한 수사는 줄이고 **핵심 사항을 간략히 bullet-point**로 정리해줄 수 있습니다. 예컨대 "1. 경력과 강점을 강조, 2. 지원 동기의 구체화, 3. 간결한 문장 사용..." 등 실행 가능한 **실용 팁 목록** 형태가 될 가능성이 높습니다. 어조는 지나치게 딱딱하기보다는 **깔끔하고 전문적인 안내**처럼 느껴질 것입니다.

이러한 역할별 응답을 실제로 비교해보면, **내용의 구성, 말투, 강조점**에서 뚜렷한 차이가 나타나는 것을 관찰할 수 있습니다. 수강생 여러분은 각 답변을 읽고, 어떤 요소들이 역할 때문에 달라졌는지 토의해 보세요. 예를 들어 튜터봇은 왜 그렇게 길고 자세하게 설명했는지, 상담사는 어떤 단어 선택으로 공감을 표현했는지 등을 생각해봅니다. 이 과정을 통해 **역할 지시 한 줄**이 얼마나 큰 변화를 가져오는지 직접 체감하는 것이 목표입니다.

4. Colab 실습: 역할별 응답 수집 및 평가

이번에는 Python 환경(Colab 노트북)을 활용하여, 위에서 수동으로 비교한 역할별 응답을 **자동으로 여러 번 수집**하고 체계적으로 **평가**해 보겠습니다. 수강생들은 이미 Python 중급 수준이며 Colab 사용에 익숙하다고 가정합니다. 아래에 안내된 코드를 Colab에 실행하여 결과를 얻고 분석해 봅시다.

4.1. OpenAI API 클라이언트 설정

먼저 OpenAI Python 라이브러리(version 1.6 이상)를 사용하여 API 클라이언트를 초기화합니다. Colab에서 `openai` 라이브러리가 설치되어 있지 않다면 `!pip install openai` 로 설치해 주세요. API 키는 Colab 환경에 미리 저장되어 있다고 가정하며, `userdata.get('OPENAI_API_KEY')` 로 불러옵니다:

##실습 2.2. Google Colab 실습

```
!pip install -U openai

from google.colab import userdata
import os
import openai

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

client = openai.OpenAI()
```

위 코드에서는 `openai.OpenAI()` 를 호출하여 클라이언트를 생성했습니다. 이제 이 클라이언트를 통해 **ChatCompletion API**를 사용하고, 모델로는 기본적으로 *GPT-4o-mini*를 활용할 것입니다 (필요시 *GPT-4o*로 변경 가능).

4.2. 역할 및 프롬프트 설정

이제 실험에 사용할 **역할별 시스템 프롬프트**와 **사용자 질문**을 정의합니다. 앞서 Playground 실습과 동일한 시나리오를 사용하겠습니다. 여러 역할의 응답을 한꺼번에 비교하기 위해, 역할별로 시스템 메시지를 딕셔너리로 정리합니다. 그리고 사용자 메시지는 공통으로 자기소개서 질문을 넣겠습니다:

##실습 2.3. Google Colab 실습

```
# 실험에 사용할 역할별 시스템 프롬프트 정의
role_system_prompts = {
```

```

"튜터봇": "당신은 글쓰기 교육 전문가입니다. 학생이 자기소개서를 잘 쓰도록 친절하게 지도해주세요.",
"상담사": "당신은 공감적인 커리어 상담사입니다. 자기소개서 작성에 부담을 느끼는 이에게 위로와 조언을 해주세요.",
"면접관": "당신은 기업 인사담당 면접관입니다. 지원자의 자기소개서를 평가하고 개선점을 알려주세요.",
"비서봇": "당신은 유능한 비서입니다. 자기소개서를 잘 쓰기 위한 요점을 간결하게 정리해서 알려주세요."
}

```

공통 사용자 질문 설정

```
user_question = "자기소개서를 잘 쓰는 법을 알려줘"
```

여기서 `role_system_prompts` 딕셔너리의 키는 역할 이름이고, 값은 그 역할을 모델에게 부여하는 시스템 메시지입니다. 예를 들어 `"튜터봇"` 키에 대응하는 메시지는 모델을 글쓰기 교육 전문가로 설정하는 내용입니다.

4.3. 역할별 응답 수집

이제 각 역할에 대해 OpenAI API를 호출하여 응답을 받아보겠습니다. **동일한 프롬프트를 반복 호출**하면서 응답이 얼마나 일관되게 나오는지도 확인하기 위해, 각 역할당 몇 번씩 호출해 보겠습니다. (예제에서는 각 역할별로 3회씩 호출합니다.) 모델의 출력을 재현 가능하게 하려면 `temperature` 값을 낮추거나 고정(random seed 개념은 지원되지 않으므로 `temperature=0`으로 deterministic하게 설정 가능)해야 하지만, 여기서는 자연스러운 변화를 보기 위해 기본 설정을 사용합니다:

##실습 2.4. Google Colab 실습

```

import pandas as pd
import time

results = [] # 결과를 담을 리스트

# 각 역할에 대해 n번씩 응답 생성
n = 3 # 반복 호출 횟수
for role_name, sys_prompt in role_system_prompts.items():
    for i in range(1, n+1):
        messages = [
            {"role": "system", "content": sys_prompt},
            {"role": "user", "content": user_question}

```

```

]
# GPT-4o-mini 모델로 ChatCompletion 생성
response = client.chat.completions.create(
    messages=messages,
    model="gpt-4o-mini"
)
answer = response.choices[0].message.content # 응답 메시지 추출
results.append({
    "role": role_name,
    "attempt": i,
    "answer": answer.strip()
})
time.sleep(1) # 1초 간격으로 요청

# 결과를 데이터프레임으로 정리
df = pd.DataFrame(results)
df.head(8) # 일부 미리보기

```

위 코드에서는 `openai.ChatCompletion.create` 를 통해 대화를 완성하고, `response.choices[0].message.content` 로 **모델의 답변 문자열**을 얻고 있습니다. 그런 다음 `results` 리스트에 역할, 시도 횟수, 답변을 저장합니다. 마지막으로 `pandas` 데이터프레임 `df` 로 변환하여 결과를 표 형태로 확인합니다.

API 호출이 완료되면, `df` 에는 각 역할별로 1~3회 차에 얻은 총 12개의 답변이 기록됩니다. 예시로 데이터프레임의 일부를 출력하면 다음과 비슷한 형식일 것입니다:

	role	attempt	answer
0	튜터봇	1	"자기소개를 잘 쓰려면 우선 자신에 대한 ... "
1	튜터봇	2	"자기소개를 효과적으로 작성하기 위해서는 ... "
2	튜터봇	3	"좋은 자기소개를 쓰는 핵심은 크게 세 가지... "
3	상담사	1	"자기소개를 쓰는 게 부담될 수 있죠. 우선 ... "
4	상담사	2	"취업 준비 과정에서 자기소개서는 고민이 ... "
...	...	...	...

(위 내용은 출력 예시를 일부 가공한 것이며, 실제 실행 시 결과와 다를 수 있습니다.)

이처럼 각 행에는 **역할(role)**, **시도 횟수(attempt)**, 그리고 **모델의 답변(answer)**이 들어갑니다.

4.4. 응답 내용 비교 및 평가

이제 데이터프레임에 모인 역할별 응답들을 비교하면서, **역할에 따른 응답의 일관성과 특성**을 평가해보겠습니다. 우선 같은 역할 내에서 여러 번 호출한 응답들을 살펴보세요. 예를 들어 튜터봇의 1~3회 응답이 **유사한 구조와 톤**을 유지하고 있는지, 상담사의 응답들이 모두 **공감적인 어조**를 일관되게 보여주는지 등을 확인합니다. 일반적으로 **시스템 프롬프트에 역할을 명확히 지정했기 때문에**, 랜덤성이 약간 개입해도 **큰 틀에서 일관된 스타일**을 유지할 것입니다. (만약 어떤 역할의 응답이 매 호출마다 크게 달라진다면, 프롬프트 지시가 모호하지 않았는지 점검해야 합니다.)

다음으로 **역할 간** 응답을 비교합니다. 이미 Playground 실습에서 한 번 비교를 했지만, 여러 번 호출된 결과를 통해 더욱 **객관적으로 공통 패턴**을 볼 수 있습니다. 특히 다음 항목들에 주목해보세요:

- **정확성:** 각 답변에 제시된 조언이나 정보가 실제 자기소개서 작성에 도움이 되는 **타당하고 정확한 내용**인가? (예: 튜터봇과 비서봇의 조언은 실제 팁 위주로 정확도가 높을 가능성이 큼.)
- **공감성:** 답변이 **사용자의 감정이나 상황에 공감**하고 있나요? (예: 상담사 역할의 답변은 이 측면에서 점수가 높고, 비서봇은 낮을 것입니다.)
- **실용성:** 제시된 조언이 **구체적이고 실행가능한지** 여부를 봅니다. (예: 비서봇과 튜터봇은 실용적 팁을 구조화해서 줄 것이고, 상담사는 다소 추상적인 격려를 포함할 수 있습니다.)

이러한 기준으로 각 역할의 답변을 서로 **평가 및 비교**해 봅니다. 필요하다면 아래처럼 데이터 프레임에 간단한 평가 컬럼을 추가해볼 수도 있습니다 (수동 평가 점수를 기입하거나, 임의로 예시 점수를 넣을 수 있습니다):

##실습 2.5. Google Colab 실습

예시: 각 역할 첫 번째 답변에 대해 임의의 평가점수를 부여 (정확성, 공감성, 실용성 각 5점 만점)

```
evaluation = {  
  "튜터봇": {"정확성": 5, "공감성": 3, "실용성": 5},  
  "상담사": {"정확성": 4, "공감성": 5, "실용성": 4},  
  "면접관": {"정확성": 5, "공감성": 2, "실용성": 5},  
  "비서봇": {"정확성": 4, "공감성": 2, "실용성": 5}  
}
```



```
# DataFrame에서 첫 번째 시도(answer)에 해당하는 행만 추려 평가 테이블 생성
eval_df = pd.DataFrame([
    {"role": role, **scores} for role, scores in evaluation.items()
])
eval_df
```

위 코드는 임의로 점수를 넣은 예시입니다. 실제로는 여러분이 각 답변을 읽고 **주관적으로 판단한 평가를 기록**할 수 있습니다. 평가 기준은 상황에 따라 다르겠지만, 여기서는 정확성(사실 및 도움 되는 내용의 정확도), 공감성(사용자 감정에 대한 이해와 배려), 실용성(바로 적용할 수 있는 구체적 조언의 정도) 등을 고려했습니다. 이런 식으로 평가표를 만들면 어떤 역할의 답변이 어떤 측면에서 강점이 있는지 한눈에 볼 수 있습니다.

##실습 2.6. Google Colab 실습

```
prompt = f"""
역할에 따른 응답의 일관성과 특성을 다음 평가항목에 대해 예시와 같이 평가(1~5, 5점 만점)를 JSON형식으로 만듭니다.
```

평가항목:

- 정확성: 각 답변에 제시된 조언이나 정보가 실제 자기소개서 작성에 도움이 되는 타당하고 정확한 내용인가? (예: 튜터봇과 비서봇의 조언은 실제 팁 위주로 정확도가 높을 가능성이 큼니다.)
- 공감성: 답변이 사용자의 감정이나 상황에 공감하고 있나요? (예: 상담사 역할의 답변은 이 측면에서 점수가 높고, 비서봇은 낮을 것입니다.)
- 실용성: 제시된 조언이 구체적이고 실행가능한지 여부를 봅니다. (예: 비서봇과 튜터봇은 실용적 팁을 구조화해서 줄 것이고, 상담사는 다소 추상적인 격려를 포함할 수 있습니다.)

예시:

```
{{
    "튜터봇": {"정확성": 5, "공감성": 3, "실용성": 5}},
    "상담사": {"정확성": 4, "공감성": 5, "실용성": 4}},
    "면접관": {"정확성": 5, "공감성": 2, "실용성": 5}},
    "비서봇": {"정확성": 4, "공감성": 2, "실용성": 5}}
}},
...
```

역할(role)에 따른 응답(answer):

```
{results}
```

```
"""
```

```
print(prompt)
```

```
response = client.chat.completions.create(
    messages=[
        {"role": "user", "content": prompt}
    ],
    model="gpt-4o-mini",
    response_format={"type": "json_object"}
)
evaluation = response.choices[0].message.content
print(evaluation)
print(type(evaluation))
```

```
import json

evals = json.loads(evaluation) # 문자열을 Dictionary로 변환

all_data = []
# Iterate directly through the items in the evals dictionary
for role, scores in evals.items():
    all_data.append({'role': role, **scores})

eval_df = pd.DataFrame(all_data)
eval_df
```

마지막으로, 수집된 응답들을 **JSON 파일로 저장**하거나 **요약**해서 볼 수도 있습니다. 예를 들어 다음과 같이 JSON으로 저장해 둘 수 있습니다:

```
import json
# 모든 결과를 JSON 파일로 저장
with open("role_prompt_responses.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
```

이렇게 하면 나중에 응답 데이터를 불러와 분석을 이어서 할 수도 있습니다. 또는 각 역할별 답변을 모델에게 요약시키는 메타프롬프트를 작성해 볼 수도 있습니다 (예: "다음은 튜터봇

의 답변 3개입니다... 공통점을 요약해줘").

이번 Colab 실습을 통해 여러분은 **프롬프트에 역할을 부여하는 방법과 그 효과**를 프로그래밍적으로 검증해보았습니다. 역할에 따라 응답 내용과 스타일이 어떻게 달라지는지, 그리고 그러한 스타일이 일정하게 유지되는지 살펴보는 것이 핵심 목표였습니다.

5. 역할 설정이 모델에 미치는 영향 분석

이제 앞서의 실습 결과와 추가 실험을 토대로, **역할 지정을 했을 때와 하지 않았을 때** 모델 응답의 차이를 정리해보겠습니다. 또한 **시스템 메시지로 역할을 설정하는 것과 일반 user 프롬프트로 지시하는 것**의 차이도 간략히 언급합니다.

- **역할 미지정 vs 역할 지정:** 아무 역할도 주지 않은 기본 상태의 모델은 대체로 **친절하고 중립적인 어조**로 답변합니다. 자기소개서 조언이라면 무난하게 일반 팁(예: 맞춤법 검사하기, 자신의 강점 강조하기 등)을 줄 것입니다. 반면 역할을 주면 답변의 **관점과 표현**이 그 역할의 성격을 반영하여 바뀝니다. 예를 들어 **튜터봇 역할**을 준 경우, 답변에 "...라고 할 수 있습니다.", "~해보세요." 등 가르치는 투의 문장이 많아졌다면, 이는 역할 지시의 영향입니다. **상담사 역할**의 경우 "이해합니다, ~할 수도 있어요."처럼 공감형 어투와 자기소개서 외적인 심리적 조언이 추가됐을 수 있습니다. 이렇듯 **같은 질문에 대해서도** 역할 유무에 따라 **답변의 내용 선택과 어조**가 크게 달라짐을 확인했습니다.
- **시스템 프롬프트 역할 vs 유저 프롬프트 역할:** 두 방법 모두 모델이 역할을 받아들일도록 할 수 있지만, **시스템 프롬프트**에 적는 방식이 보다 **권장**됩니다. 시스템 메시지는 모델이 대화를 시작하기 전에 항상 참조하는 지침이므로, 역할을 강하게 인지시키고 **대화 내내 유지**시키는 효과가 있습니다. 반면 유저 메시지에 "당신은 ~~ 역할이다"라고 적으면 모델이 그 지시를 따르긴 하지만, 대화가 길어질수록 초기 역할 지시가 맥락 속에서 묻혀버릴 수 있습니다. 또한 시스템 메시지를 쓰면 유저가 불필요한 역할 설명을 매번 입력하지 않아도 되어 **프롬프트가 깔끔해지는** 장점이 있습니다. 이번 실습에서도 시스템 메시지로 역할을 줬을 때 모델이 일관되게 해당 역할을 연기함을 볼 수 있었습니다. (실험으로, 동일한 역할 지시를 시스템 대신 유저 메시지에 넣어 답변을 비교해보면 내용은 비슷하지만, 모델이 간혹 자신의 역할을 언급하는 방식 등이 약간 다를 수 있습니다. 예: 시스템 지시의 경우 답변에서 "제가 도움을 드리겠습니다..."라고 하고, 유저 지시의 경우 "~~ 전문가로서 말씀드리면..."처럼 스스로 역할을 언급하기도 합니다.)
- **역할 지정에 따른 응답 품질:** 역할을 잘 활용하면 **사용자 의도에 부합하는 맞춤형 답변**을 얻을 수 있지만, 응답의 **사실적 정확도**는 결국 모델의 지식과 논리에 달려있습니다. 역할 그 자체가 모델의 지식을 바꾸지는 않지만, **어떤 역할이냐에 따라 답변 내용의 우선순위가 바뀌어** 결과적으로 유용도가 영향받을 수 있습니다. 예를 들어, 자기소개서 조언에서 상담사 역할은 동기부여에 힘쓰는 반면 구체적인 글쓰기 팁은 부족할 수 있고, 비서 역할은 팁은 풍부하지만 정서적 지원은 없겠지요. 따라서 **해결하려는 과제에 적합한 역할**을 선택하는 것이 중요합니다. 한편, 최근 연구에서는 다양한 질문 세트에 대해 역할을 추가

한 경우와 아닌 경우 성능을 비교했는데, **전체적으로는 큰 차이가 없거나 일부는 오히려 감소하기도** 했다고 합니다. 이는 특히 **사실 질문**의 경우로, 어떤 페르소나를 갖든 정답은 바뀌지 않기 때문으로 해석합니다. 반대로, **창의적인 응답이나 단계적 추론**이 필요한 경우에는 역할 (예: 비유를 잘하는 이야기꾼, 논리를 중시하는 분석가 등)을 통해 답변 전개 방식에 차이를 주어 **특정 측면의 품질을 향상**시키는 전략이 유용할 수 있습니다.

- **역할 유지 및 전환:** 모델은 일반적으로 시스템에 주어진 역할을 대화 내내 유지합니다. 다만 새로운 시스템 메시지를 보내거나, 사용자가 다른 역할을 새로 지시하면 **역할이 전환** 될 수 있습니다. 여러 가지 역할을 연속으로 실험할 때는 각 대화를 별개 세션으로 초기화하거나, 매번 역할 지시를 다시 확실히 해주는 것이 안전합니다. 한 세션 안에서 역할을 바꾸면 혼선이 생길 수 있으므로 지양합니다.

요약하면, 역할 기반 프롬프트는 **응답의 스타일과 관점**을 제어하는 강력한 도구지만, **응답의 근본적인 정확성**을 보장해주지는 않습니다. 그렇지만 사용자 질문에 가장 알맞은 **맥락과 톤**을 잡아줌으로써 답변을 **더 이해하기 쉽게** 또는 **더 사용자 친화적으로** 만들 수 있다는 점에서, 챗봇 개발이나 AI 활용 측면에서 큰 가치가 있습니다.

6. 역할 프롬프트 활용 팁 및 기대 효과

역할 기반 프롬프트 설계와 그에 따른 **응답 변화**를 깊이 있게 탐구해보았습니다. 이제 다음과 같은 역량을 갖추었을 것입니다:

- **역할 프롬프트의 개념 이해:** 모델에 역할을 부여하면 어떤 식으로 응답이 달라지는지 원리를 이해했습니다. 단순히 정보 전달을 넘어, AI의 **페르소나를 설정**함으로써 사용자에게 더욱 몰입감 있고 유용한 상호작용을 제공할 수 있습니다.
- **전략적 역할 설계 능력:** 프롬프트 안에 어떤 역할 지시를 넣을지, 그리고 그것을 시스템 메시지로 줄지 유저 메시지로 줄지 등 **프롬프트 설계 전략**을 연습했습니다. 이제 과제에 맞는 **적절한 역할을 선정**하고, 구체적인 말투와 형식까지 설계하여 원하는 응답을 이끌어낼 수 있을 것입니다.
- **응답 비교 및 평가 경험:** Playground와 Colab 실습을 통해 역할별 모델 응답을 직접 비교하고 **분석하는 방법**을 배웠습니다. 특히 Python을 활용해 여러 응답을 수집하고, 이를 테이블로 정리하여 **일관성, 특성, 품질**을 평가해 본 경험은, 추후 프롬프트 튜닝을 체계적으로 하는 데 도움이 될 것입니다.
- **모델 행위 이해 심화:** 다양한 역할을 실험하면서, 모델의 언어 생성 행위에 대한 **감각**이 더욱 깊어졌을 것입니다. 예를 들어 어떤 역할 지시어가 모델의 어투에 미치는 영향, 응답 내용의 편향 가능성 등도 체감했을 것입니다. 이를 바탕으로, 실무에서 챗봇을 설계할 때 **역할 프롬프트를 적절히 활용하거나 피해야 할 상황**을 구분할 수 있게 됩니다 (예: 고객 지원 봇에는 친절한 상담사 역할, 지식 Q&A 봇에는 전문 분야 전문가 역할 등).

활용 팁: 실제로 프롬프트 엔지니어링을 할 때 역할 지정을 잘 활용하면, 추가적인 예시나 복잡한 지시 없이도 **원하는 스타일의 답변**을 얻는 데 큰 도움이 됩니다. 예를 들어 **요약 작업**을 시킬 때도 그냥 요약해 달라는 것보다 "*당신은 세계적인 소설가입니다. 문체를 유지하며 다음 글을 간략히 요약하세요.*"처럼 역할을 주면 문학적으로 세련된 요약을 받을 수 있습니다. 다만 앞서 언급한대로 역할 설정이 **편향**이나 **고정관념**을 강화할 수 있다는 점을 명심해야 합니다. 역할 묘사는 독립적으로 하며, 필요한 경우 한 번 응답을 받은 후 추가 프롬프트로 "*보다 객관적으로 설명해줘*"처럼 **조정 프롬프트**를 이어가면서 최적의 답변을 끌어내는 것이 좋습니다.